VOLUME 6 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 6

EXCEPTION HANDLING & FILE HANDLING

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 6

About This Module

Welcome to Volume 6 of the IT Study Hub Complete Java Programming Series. Until now, we have assumed our code operates in a perfect world: users always enter valid data, arrays are always properly sized, and resources are always available. However, in the real world, **things go wrong**.

Users type text when you ask for numbers. Internet connections drop. Files you try to open might not exist. If you don't anticipate these events, your software will crash. This module introduces **Exception Handling**—the art of catching errors before they crash your program. Furthermore, you will finally break out of the temporary console and learn **File Handling** to permanently save your data to the hard drive.

Learning Outcomes

- Differentiate between compile-time Errors, runtime Errors, and Exceptions.
- Implement robust safety nets using `try`, `catch`, and `finally` blocks.
- Understand the Exception Hierarchy and differentiate Checked vs. Unchecked Exceptions.
- Delegate error responsibility using `throws` and intentionally trigger errors using `throw`.
- Create and utilize Custom Exceptions tailored to your application's domain logic.
- Read from and Write to external text files using Java's I/O Streams and Readers/Writers.
- Understand the fundamentals of Object Serialization.

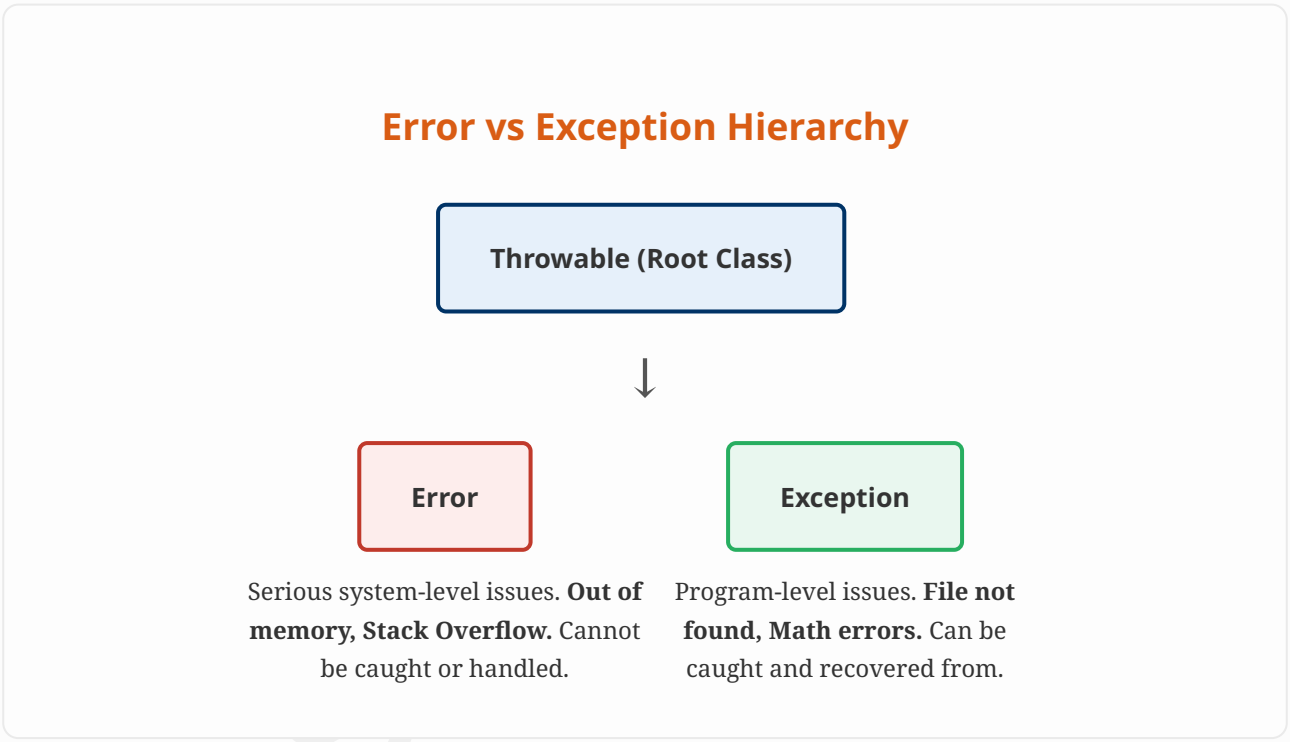
Module Coverage

We begin by dissecting the anatomy of a crash and how to intercept it. You will write code that safely handles division by zero and bad array indexing. Next, we explore File and Path objects, transitioning into high-performance reading and writing using `BufferedReader` and `BufferedWriter`. We cap off the module with a "File-Based Student Record Management System" that permanently stores data on your hard drive, fortified by custom exception handling.

IT STUDY HUB

1. Introduction to Exceptions

An exception is an unwanted or unexpected event occurring during the execution of a program (at runtime) that disrupts the normal flow of instructions. Exception Handling is a mechanism to handle these runtime errors so that the normal flow of the application can be maintained.



Concept: What is a try-catch block?

A safety net that attempts risky code and 'catches' any resulting crashes before they terminate the program.

> ***Real-Life Analogy:***
> *Imagine a trapeze artist at a circus. The 'try' block is the dangerous trapeze act. If everything goes perfectly, great! But if they fall (an exception occurs), the 'catch' block is the safety net underneath that catches them so they survive and the show can continue.*

> **Simple Explanation:**
> You wrap any 'risky' code inside a `try {}` block. Immediately following it, you provide a `catch {}` block. If the risky code throws an error, Java immediately aborts the `try` block and jumps straight into the `catch` block, allowing you to print an apology or fix the issue without crashing the entire app.

Syntax / Structure:

```
try {
    // Risky code that might crash
} catch (ExceptionType e) {
    // Code to run IF a crash occurs
}
```

Example Code:

```
import java.util.Scanner;

public class DivisionSafely {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter numerator: ");
        int num = scan.nextInt();
        System.out.print("Enter denominator: ");
        int den = scan.nextInt();

        try {
            // Risky code: Denominator might be 0!
            int result = num / den;
            System.out.println("Result: " + result);
```

```
        } catch (ArithmeticException e) {
            // This ONLY runs if a math error happens
            System.out.println("CRASH PREVENTED: Cannot divide by zero!");
        }

        System.out.println("Program continues normally...");
        scan.close();
    }
}
```

```
Expected Output:
Enter numerator: 10
Enter denominator: 0
CRASH PREVENTED: Cannot divide by zero!
Program continues normally...
```

Code Explanation:
Without `try-catch`, entering 0 for the denominator would crash the program entirely with an `ArithmeticException`, and 'Program continues normally...' would NEVER print. By catching it, we gracefully handled the user's mistake.

> ***Real-world Use:***
> *Connecting to a web server. If the server is offline, the app shouldn't crash and close; it should catch the error and show a 'No Internet Connection' pop-up to the user.*

> **Common Mistakes:**
> - Putting completely safe, non-crashing code inside a try-catch block. It adds unnecessary performance overhead to the JVM.

> **Best Practices:**
> - Keep your `try` blocks as small as possible. Only wrap the exact lines that pose a threat.

Practice Task:
Write a program with an array of size 3. Put an attempt to print `array[5]` inside a try block, and catch the `ArrayIndexOutOfBoundsException`.

IT STUDY HUB

Concept: The 'finally' Block

A block of code that ALWAYS executes, regardless of whether an exception occurred or not.

> ***Real-Life Analogy:***
> *If you rent a hotel room, you might have a great vacation (try succeeds), or you might get sick and have a terrible time (exception caught). Regardless of what happened, you MUST return the room key and check out (finally) before leaving.*

> **Simple Explanation:**
> When dealing with external resources (like reading a file or opening a database), you use memory that must be released. The `finally` block guarantees that the cleanup code will run, even if the `try` block crashed or the `catch` block threw another error.

Syntax / Structure:

```
try {
    // Risky code
} catch (Exception e) {
    // Handle error
} finally {
    // ALWAYS runs (cleanup code)
}
```

Example Code:

```
import java.util.Scanner;

public class FinallyExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.println("Inside Try...");
            int data = 100 / 0; // Will cause an exception
            System.out.println("This line is skipped!");
        } catch (ArithmeticException e) {
            System.out.println("Inside Catch: Error intercepted.");
        } finally {
```

```
            System.out.println("Inside Finally: Closing resources.");
            sc.close(); // Perfect place to close scanners or files!
        }

        System.out.println("End of program.");
    }
}
```

```
Expected Output:
Inside Try...
Inside Catch: Error intercepted.
Inside Finally: Closing resources.
End of program.
```

Code Explanation:
The code hits the `/ 0` and jumps to `catch`. After `catch` finishes, it moves to `finally`. Even if the `catch` block had a `return;` statement trying to end the method early, Java forces the `finally` block to execute before actually leaving.

> ***Real-world Use:***
> *Closing File Readers or terminating SQL Database connections to prevent memory leaks.*

> **Common Mistakes:**
>
> - Assuming `finally` only runs if there's an error. It ALWAYS runs (unless you forcibly kill the JVM with `System.exit(0)`).

> **Best Practices:**
>
> - Use `finally` strictly for resource cleanup (closing files, scanners, network sockets). Don't put core business logic in there.

Practice Task:
Write a try-catch-finally block where the try block successfully prints 'Hello', the catch block is empty, and finally prints 'Goodbye'. Observe the output.

---

try-catch-finally Execution Flow

Scenario	try block	catch block	finally block
No Exception Occurs	**Runs completely**	Skipped	**Runs**
Exception Occurs & Caught	Runs until error, then aborts	**Runs**	**Runs**

IT STUDY HUB

2. Advanced Exception Handling

IT STUDY HUB

Concept: Multiple Catch Blocks

Handling different types of exceptions in distinct ways.

> ***Real-Life Analogy:***
> *A hospital emergency room. If a patient comes in with a broken bone, they go to the Orthopedics room (Catch block 1). If they have a rash, they go to Dermatology (Catch block 2). You don't treat every emergency the exact same way.*

> **Simple Explanation:**
> A single `try` block might contain code that could trigger different errors (e.g., a math error OR an array error). You can stack multiple `catch` blocks. Java evaluates them from top to bottom, looking for the first one that matches the specific error thrown.

Syntax / Structure:

```
try {
    // Code
} catch (ArithmeticException e) {
    // Handle math error
} catch (ArrayIndexOutOfBoundsException e) {
    // Handle array error
} catch (Exception e) {
    // Generic fallback for any other error
}
```

Example Code:

```
public class MultiCatch {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};

        try {
            // Risky line 1: Array Bounds
            System.out.println("Number: " + numbers[5]);

            // Risky line 2: Math (This won't run because line 1 crashes first)
            int result = numbers[0] / 0;

        } catch (ArithmeticException e) {
```

```
            System.out.println("Math Error: Division by zero.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Error: Index does not exist.");
        } catch (Exception e) {
            System.out.println("Unknown Error: " + e.getMessage());
        }
    }
}
```

```
Expected Output:
Array Error: Index does not exist.
```

Code Explanation:
The code crashes on `numbers[5]`. Java ignores the `ArithmeticException` block because it doesn't match the error type. It successfully matches the `ArrayIndexOutOfBoundsException` block and runs it. The generic `Exception e` at the bottom acts as a global safety net.

> ***Real-world Use:***
> *When downloading a file from the internet, you need different error messages for 'No WiFi' versus 'File Corrupted'.*

Common Mistakes:

- Putting the generic `catch (Exception e)` at the very TOP of the stack. Because it is the parent of all exceptions, it will catch everything, making the specific catch blocks below it 'Unreachable Code' (a compiler error).

Best Practices:

- Always order catch blocks from most specific to most generic (Child exceptions first, Parent exception last).

Practice Task:
Create an array of size 2. Ask the user for an index, and for a divisor. Catch both array and math exceptions in separate blocks.

Checked vs. Unchecked Exceptions

Feature	Checked Exceptions	Unchecked Exceptions
When are they checked?	At **Compile-time**. The compiler forces you to handle them.	At **Run-time**. The compiler does no handle them.
Inheritance	Direct children of `Exception`.	Children of `RuntimeException`.
Cause	External factors outside your control (e.g., File missing, network down).	Logical programming mistakes (e.g zero, null objects).
Examples	`IOException`, `SQLException`, `ClassNotFoundException`	`ArithmeticException`, `NullPointerException`

IT STUDY HUB

3. The 'throw' and 'throws' Keywords

IT STUDY HUB

Concept: Delegating with 'throws'

Declaring that a method MIGHT trigger a Checked Exception, and forcing whoever calls the method to handle it.

> ***Real-Life Analogy:***
> *A construction worker finds a live bomb on site. Instead of trying to defuse it themselves (using a try-catch), they shout 'BOMB!' (throws) to alert the bomb squad (the calling method) to come and handle it.*

> **Simple Explanation:**
> When dealing with Checked Exceptions (like reading files), Java forces you to acknowledge the risk. If you don't want to write a `try-catch` inside your method, you can add `throws ExceptionName` to the method signature. This passes the buck. The method that called you is now legally required to handle it.

Syntax / Structure:

```
public void readFile() throws IOException {
    // Risky file reading code without try-catch
}
```

Example Code:

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class ThrowsExample {

    // Method delegates responsibility using 'throws'
    public static void connectToFile() throws IOException {
        File file = new File("missing_file.txt");
        FileReader fr = new FileReader(file); // Triggers a Checked Exception
    }

    public static void main(String[] args) {
        try {
            System.out.println("Attempting connection...");
            connectToFile(); // The Caller MUST handle it!
```

```
        } catch (IOException e) {
            System.out.println("Handled in main: File was not found.");
        }
    }
}
```

```
Expected Output:
Attempting connection...
Handled in main: File was not found.
```

Code Explanation:
`FileReader` throws a checked `IOException`. The `connectToFile` method doesn't want to handle it, so it `throws` it up to `main`. `main` wraps the call in a `try-catch` to cleanly intercept the passed-up error.

Real-world Use:
Utility methods (like a generic database connector) shouldn't decide what the UI error message is. They `throw` the error up to the UI controller so the UI can show the correct pop-up alert.

Common Mistakes:

- Adding `throws` to the `main` method. If `main` throws the exception, it goes to the JVM, which immediately crashes the application. Never throw from main!

Best Practices:

- Use `throws` when the method doesn't have enough context to properly handle the error, and the caller is better equipped to decide what to do.

Concept: Intentionally triggering with 'throw'

Manually creating and triggering an exception object when business logic rules are violated.

> ***Real-Life Analogy:***
> *A football referee blowing a whistle. Even if no players are physically injured (no standard Java exception), if a player breaks a game rule (e.g., touches the ball with their hands), the referee manually throws a penalty flag.*

> **Simple Explanation:**
> Sometimes Java thinks code is fine, but it violates your application's rules. For example, age can technically be -5 in math, but not in real life. We use the `throw new` keywords to intentionally crash the process and generate an error message.

Syntax / Structure:

```
if (condition_is_bad) {
    throw new ExceptionType("Custom Error Message");
}
```

Example Code:

```
public class ThrowExample {

    public static void validateAge(int age) {
        if (age < 18) {
            // Manually triggering an exception!
            throw new IllegalArgumentException("Access Denied: Must be 18 or older.");
        } else {
            System.out.println("Access Granted: Welcome!");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (IllegalArgumentException e) {
            // e.getMessage() retrieves the custom string we passed
            System.out.println("Validation Error -> " + e.getMessage());
```

```
        }
    }
}
```

```
Expected Output:
Validation Error -> Access Denied: Must be 18 or older.
```

Code Explanation:
The `validateAge` method evaluates the input. Since 15 < 18, it executes `throw new IllegalArgumentException(...)`. This instantly stops the method and throws the newly created error object down to the `catch` block in `main`.

Real-world Use:
Validating user inputs in banking apps. `if (withdrawAmount > balance) throw new InsufficientFundsException();`

Common Mistakes:

- Confusing `throw` (verb used inside the method body to create an error) with `throws` (noun used in the method signature to declare a risk).

Best Practices:

- Provide clear, highly descriptive string messages when throwing exceptions to make debugging easier for other developers.

Practice Task:
Create a method that accepts a password string. If the length is < 6, throw an IllegalArgumentException saying 'Password too short'.

throw vs. throws

Feature	throw	throws
Purpose	Used to manually *trigger* an exception explicitly.	Used to *declare* that a method might trigger an exception.
Location	Inside the method body.	In the method signature.
Syntax	`throw new Exception("Msg");`	`public void myMethod() throws Exception`
Quantity	Can only throw one exception at a time.	Can declare multiple exceptions (comma separated).

IT STUDY HUB

4. Creating Custom Exceptions

IT STUDY HUB

Concept: Custom Exception Classes

Creating your own personalized Exception types that fit your specific application's business rules.

> ***Real-Life Analogy:***
> *Instead of using a generic 'Check Engine' light on a car dashboard (generic Exception), you wire up a highly specific 'Low Tire Pressure' light (Custom Exception) so the mechanic instantly knows the context.*

> **Simple Explanation:**
> You can create a custom exception simply by writing a new Class that `extends Exception` (for Checked) or `extends RuntimeException` (for Unchecked). You give it a constructor that passes your custom error message up to the parent using `super()`.

Syntax / Structure:

```
class MyCustomException extends Exception {
    public MyCustomException(String message) {
        super(message);
    }
}
```

Example Code:

```
// 1. Define the Custom Exception
class InvalidVoterException extends Exception {
    public InvalidVoterException(String message) {
        super(message); // Passes msg to the Exception parent class
    }
}

public class VotingSystem {
    // 2. Declare it using 'throws'
    public static void checkVoter(int age) throws InvalidVoterException {
        if (age < 18) {
            // 3. Trigger it using 'throw'
            throw new InvalidVoterException("Age " + age + " is too young to vote in Ind
        }
        System.out.println("Voter approved.");
```

```
    }

    public static void main(String[] args) {
        try {
            checkVoter(16);
        } catch (InvalidVoterException e) { // 4. Catch your custom type!
            System.out.println("SYSTEM ALERT: " + e.getMessage());
        }
    }
}
```

```
Expected Output:
SYSTEM ALERT: Age 16 is too young to vote in India.
```

Code Explanation:
We built an entire end-to-end custom error flow. We defined the `InvalidVoterException` class. We used `throws` in the method signature, `throw new` in the logic, and caught it securely in the `main` method.

> ***Real-world Use:***
> *Enterprise software is filled with custom exceptions: `UserNotFoundException`, `InvalidTransactionException`, `DatabaseTimeoutException`.*

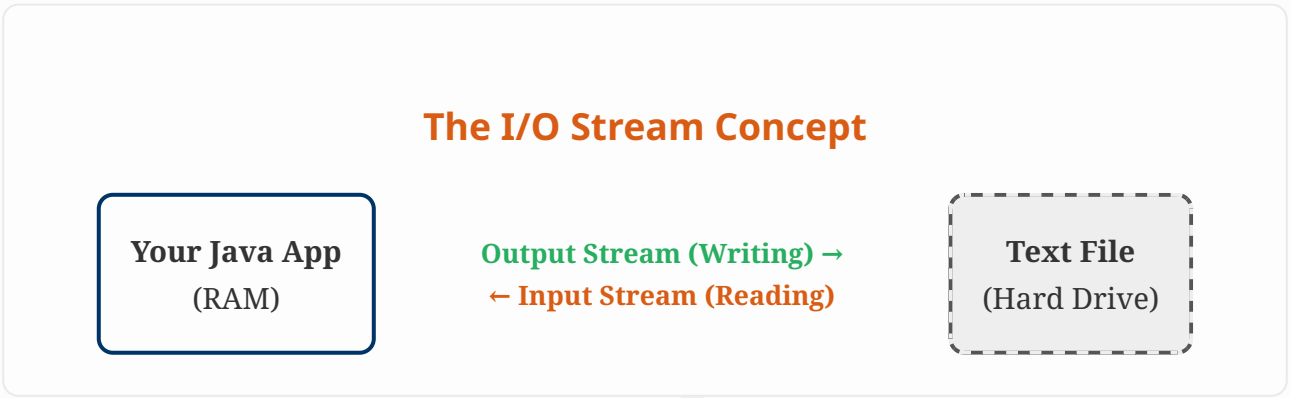
> **Common Mistakes:**
>
> - Forgetting to write the constructor with `super(message)` in the custom exception class. Without it, `e.getMessage()` will print null.

> **Best Practices:**
>
> - Suffix your class name with 'Exception' so it is immediately obvious to other developers what the class does (e.g., `DataException`, not just `DataError`).

---

5. Introduction to File Handling

Variables and Arrays store data in RAM. When the program ends, RAM is wiped clean, and your data is lost forever. To achieve **Data Persistence**, we must save data to the hard drive using Files. Java uses the `java.io` and `java.nio` packages for File I/O (Input/Output).



IT STUDY

Concept: The 'File' Class

Used to represent file and directory pathnames in a machine-independent manner.

> ***Real-Life Analogy:***
> *The `File` class is like a physical envelope. It doesn't actually contain the letter (the text) yet, and it can't read or write the letter. It just represents the location and existence of the envelope on the desk.*

> **Simple Explanation:**
> Creating a `File` object does NOT create a file on your hard drive. It just creates a reference to a path. You use it to check if a file exists, get its size, or create a new empty file.

Syntax / Structure:

```
import java.io.File;
File myFile = new File("document.txt");
```

Example Code:

```
import java.io.File;
import java.io.IOException;

public class FileBasics {
    public static void main(String[] args) {
        // 1. Create a reference to a file path
        File doc = new File("itstudyhub_notes.txt");

        try {
            // 2. Actually create the file on the hard drive
            if (doc.createNewFile()) {
                System.out.println("File created successfully: " + doc.getName());
            } else {
                System.out.println("File already exists at: " + doc.getAbsolutePath());
            }

            // 3. Use File metadata methods
            System.out.println("Is Writable? " + doc.canWrite());
            System.out.println("File Size: " + doc.length() + " bytes");
```

```
        } catch (IOException e) {
            System.out.println("An error occurred creating the file.");
        }
    }
}
```

```
Expected Output:
File created successfully: itstudyhub_notes.txt
Is Writable? true
File Size: 0 bytes
```

Code Explanation:
`new File()` sets up the reference. `doc.createNewFile()` actually asks the OS to write an empty text file to the disk. Because interacting with the hard drive might fail (e.g., disk full, no permissions), Java forces us to wrap it in a `try-catch` for `IOException`.

> ***Real-world Use:***
> *Checking if an image exists in a directory before attempting to load it onto a website dashboard.*

Common Mistakes:

- Assuming `new File("data.txt")` creates the file. It does not! You must call `.createNewFile()`.

Best Practices:

- Use relative paths (e.g., `"data.txt"`) rather than absolute paths (`"C:/Users/Admin/data.txt"`) so your program works on any computer, not just yours.

Practice Task:
Write a program that takes a filename as input, checks if it exists using `doc.exists()`, and prints the result.

---

6. Writing and Reading Text Files

To manipulate the actual text inside a file, we use specialized Stream and Reader/Writer classes. Modern Java favors **BufferedWriter** and **BufferedReader** because they read/write large chunks of text at once, making them incredibly fast.

IT STUDY HUB

Concept: Writing to a file (FileWriter & BufferedWriter)

Opening a file stream and pushing text data from RAM onto the hard drive.

> ***Real-Life Analogy:***
> *FileWriter is a small hose dripping water drop-by-drop. BufferedWriter is a massive bucket. You fill the bucket instantly in RAM, and then dump the whole bucket onto the hard drive at once. It's vastly more efficient.*

> **Simple Explanation:**
> We wrap a `FileWriter` inside a `BufferedWriter`. By default, FileWriter overwrites the entire file. If we pass `true` as the second parameter, it enters 'Append Mode', adding new text to the bottom instead of destroying old data.

Syntax / Structure:

```
FileWriter fw = new FileWriter("file.txt", true); // true = Append Mode
BufferedWriter bw = new BufferedWriter(fw);
bw.write("Text");
bw.close(); // Crucial!
```

Example Code:

```
import java.io.FileWriter;
import java.io.BufferedWriter;
import java.io.IOException;

public class WriteFileDemo {
    public static void main(String[] args) {
        // True enables 'Append Mode'
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("database.txt", true)

            bw.write("Student: John Doe");
            bw.newLine(); // Safely adds a line break for any OS
            bw.write("Grade: A");
            bw.newLine();
            bw.write("---");
            bw.newLine();

            System.out.println("Data successfully written to database.txt!");
```

```
        } catch (IOException e) {
            System.out.println("Error writing to file.");
        }
    }
}
```

```
Expected Output:
Data successfully written to database.txt!
```

Code Explanation:
We used a special syntax called **try-with-resources** `try (BufferedWriter bw = ...)`. This modern Java feature automatically closes the file the moment the try block finishes, so we don't have to manually write `bw.close()` in a `finally` block! It is extremely clean.

Real-world Use:
Saving application log files. Every time an error occurs, the server appends a new line to `error_logs.txt`.

Common Mistakes:

- Forgetting to close the writer. If you don't call `.close()`, the data stays in RAM and never actually saves to the hard drive, resulting in an empty file!

Best Practices:

- Always use `try-with-resources` to ensure writers are flushed and closed automatically, preventing memory leaks and locked files.

Practice Task:
Write a program that uses a loop to write the numbers 1 to 10 on separate lines inside a file called 'numbers.txt'.

Concept: Reading from a file (FileReader & BufferedReader)

Opening a file stream and pulling text data from the hard drive into RAM.

> ***Real-Life Analogy:***
> *A BufferedReader reads an entire page of a book into its brain at once, then hands you one sentence at a time when you ask. This prevents the slow process of asking the hard drive for data letter-by-letter.*

> **Simple Explanation:**
> We use a `BufferedReader` to `.readLine()`. It fetches a whole line of text as a String. When there are no more lines left in the file, it returns `null`.

Syntax / Structure:

```
BufferedReader br = new BufferedReader(new FileReader("file.txt"));
String line = br.readLine();
```

Example Code:

```
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class ReadFileDemo {
    public static void main(String[] args) {
        System.out.println("--- FILE CONTENTS ---");

        // try-with-resources automatically closes the reader
        try (BufferedReader br = new BufferedReader(new FileReader("database.txt"))) {

            String currentLine;
            // Read lines until there is nothing left (null)
            while ((currentLine = br.readLine()) != null) {
                System.out.println(currentLine);
            }

        } catch (IOException e) {
            System.out.println("File not found or cannot be read!");
        }
```

```
        System.out.println("--- END OF FILE ---");
    }
}
```

```
Expected Output:
--- FILE CONTENTS ---
Student: John Doe
Grade: A
---
--- END OF FILE ---
```

Code Explanation:
The `while` loop is the core pattern of file reading. `(currentLine = br.readLine()) != null` does two things at once: it fetches the next line into the variable, and checks if it's not null. It loops, printing every line, until the end of the document is reached.

> ***Real-world Use:***
> *Loading configuration settings or saved game data when an application boots up.*

> **Common Mistakes:**
> • Trying to read a file that doesn't exist without a try-catch, causing a `FileNotFoundException` crash.

> **Best Practices:**
> • Always assign `.readLine()` to a variable in the while loop condition. If you call `.readLine()` multiple times inside the loop body, you will accidentally skip lines.

Practice Task:
Create a text file manually. Write a Java program that reads the file and counts how many lines are in it.

Byte Streams vs Character Streams (Brief Overview)

Java provides two distinct parallel tracks for handling I/O operations.

Feature	Character Streams (Readers/Writers)	Byte Streams (Input/Output Streams)
Data Type	Reads human-readable text (16-bit characters).	Reads raw binary data (8-bit bytes).
Classes	`FileReader`, `FileWriter`, `BufferedReader`	`FileInputStream`, `FileOutputStream`
Use Case	.txt files, CSV files, JSON/XML documents.	Images (.png), Audio (.mp3), Video, Executables.

IT STUDY HUB

7. Basics of Object Serialization

IT STUDY HUB

Concept: Serialization & Deserialization

Converting a live Java Object into a raw stream of bytes to save to a file, and restoring it later.

> ***Real-Life Analogy:***
> *Serialization is like taking a complex Lego castle (an Object), breaking it down, and putting it into a box (a file) with an instruction manual. Deserialization is opening the box later and magically rebuilding the exact same Lego castle instantly.*

> **Simple Explanation:**
> Normally, writing to a text file requires manually extracting data (`writer.write(student.getName())`). Serialization uses `ObjectOutputStream` to instantly freeze an entire object and dump it into a `.dat` or `.ser` file. The class MUST implement the `Serializable` interface to give Java legal permission to do this.

Syntax / Structure:

```
// The marker interface
class Student implements Serializable { ... }

// Serializing (Saving)
ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("obj.ser"));
out.writeObject(studentObj);
```

Example Code:

```
import java.io.*;

// 1. Must implement Serializable! (It's an empty marker contract)
class Profile implements Serializable {
    String username;
    // transient keyword prevents this specific field from saving!
    transient String password;

    public Profile(String u, String p) {
        this.username = u;
        this.password = p;
    }
```

```
}

public class SerializationDemo {
    public static void main(String[] args) {
        Profile p1 = new Profile("AlexM", "SuperSecret123");

        // SERIALIZATION (Saving Object)
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("profi
            out.writeObject(p1);
            System.out.println("Profile object saved as binary file!");
        } catch (IOException e) { e.printStackTrace(); }

        // DESERIALIZATION (Loading Object)
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("profile.s
            Profile loaded = (Profile) in.readObject(); // Requires casting!
            System.out.println("Loaded Username: " + loaded.username);
            System.out.println("Loaded Password: " + loaded.password); // Will be null!
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

```
Expected Output:
Profile object saved as binary file!
Loaded Username: AlexM
Loaded Password: null
```

Code Explanation:
We saved the whole `Profile` object instantly. Notice the `transient` keyword! Because we marked `password` as transient, the Serializer intentionally ignored it to maintain security. When deserialized, the password loaded as `null`.

> ***Real-world Use:***
> *Saving the entire state of a video game (player location, inventory, health) to a save file, and loading it exactly as it was the next day.*

> **Common Mistakes:**
>
> - Forgetting to add `implements Serializable` to the class. It will trigger a `NotSerializableException` when you attempt to save.

Best Practices:

- Always use the `transient` keyword for highly sensitive data (passwords, social security numbers) to ensure they are never saved into physical files.

IT STUDY HUB

8. Project: File-Based Student Record System

Let's fuse custom exception handling, loops, and file I/O. We will create an application that adds students to a file and reads them back. We will use a custom Exception to block invalid data.

```
import java.io.*;
import java.util.Scanner;

// 1. Custom Exception
class InvalidGradeException extends Exception {
    public InvalidGradeException(String msg) { super(msg); }
}

public class FileDatabaseSystem {

    // 2. Method to Write to File (Delegates IO errors via 'throws')
    public static void addRecord(String name, int grade) throws IOException, InvalidGrad
        // Business Logic Validation
        if (grade < 0 || grade > 100) {
            throw new InvalidGradeException("Grade must be between 0 and 100.");
        }

        // File Writing (Append Mode)
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("students.txt", true)
            bw.write(name + "," + grade);
            bw.newLine();
            System.out.println("Success: " + name + " saved to database.");
        }
    }

    // 3. Method to Read File
    public static void displayRecords() {
        System.out.println("\n--- DATABASE RECORDS ---");
        File file = new File("students.txt");

        if (!file.exists()) {
            System.out.println("Database empty. No records found.");
            return;
        }
```

```
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                // Split the string at the comma
                String[] data = line.split(",");
                System.out.println("Student: " + data[0] + " | Grade: " + data[1]);
            }
        } catch (IOException e) {
            System.out.println("Critical Error reading database.");
        }
    }

    // 4. Main Menu Loop
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        while (true) {
            System.out.println("\n[1] Add Record  [2] View Records  [0] Exit");
            System.out.print("Choice: ");
            int choice = scan.nextInt();
            scan.nextLine(); // Consume newline

            if (choice == 0) {
                System.out.println("System shutting down...");
                break;
            } else if (choice == 1) {
                System.out.print("Enter Name: ");
                String n = scan.nextLine();
                System.out.print("Enter Grade (0-100): ");
                int g = scan.nextInt();

                // 5. Try-Catch handling the delegated exceptions
                try {
                    addRecord(n, g);
                } catch (InvalidGradeException e) {
                    System.out.println("VALIDATION ERROR: " + e.getMessage());
                } catch (IOException e) {
                    System.out.println("SYSTEM ERROR: Could not access file.");
                }

            } else if (choice == 2) {
                displayRecords();
            } else {
```

```
                System.out.println("Invalid choice.");
            }
        }
        scan.close();
    }
}
```

```
[1] Add Record  [2] View Records  [0] Exit
Choice: 1
Enter Name: Alex
Enter Grade (0-100): 105
VALIDATION ERROR: Grade must be between 0 and 100.

[1] Add Record  [2] View Records  [0] Exit
Choice: 1
Enter Name: Alex
Enter Grade (0-100): 95
Success: Alex saved to database.

[1] Add Record  [2] View Records  [0] Exit
Choice: 2
--- DATABASE RECORDS ---
Student: Alex | Grade: 95
```

Step-by-Step Execution:

1. **Custom Error:** We defined `InvalidGradeException` to represent our business rule.
2. **Writing Logic:** `addRecord` uses `throws` to push error handling to the menu. It first validates the grade (throwing our custom exception if it fails). If safe, it uses `BufferedWriter` to append data in a CSV format (`name,grade`).
3. **Reading Logic:** `displayRecords` safely checks if the file exists first. It then reads line by line. We use `line.split(",")` to split the text "Alex,95" into an array `["Alex", "95"]` for beautiful formatting.
4. **Safety Net:** In `main()`, the `try-catch` block safely catches the custom validation error, preventing a crash and allowing the user to simply try again.

---

9. Module Assessment and Interview Prep

Test your mastery of Exception and File Handling to prepare for academia and industry interviews.

A. Practice Questions (Test Your Logic)

1. Write a `try-catch` block that intentionally creates an `ArrayIndexOutOfBoundsException` and catches it, printing a friendly error message.
2. Create a program that divides two numbers. Add a `finally` block that prints "Calculation attempted". Ensure it prints whether the math succeeds or fails.
3. Write a Custom Exception class named `InvalidPasswordException`. Throw it if a given string is less than 8 characters long.
4. Use `File` to create a new file named `test.txt`. Write code to print its absolute path using `getAbsolutePath()`.
5. Write a program using `BufferedWriter` to write your name and age on two separate lines in a text file.
6. Write a program using `BufferedReader` to read the file created in the previous question and print it to the console.
7. Create a class `Car` that implements `Serializable`. Instantiate it, serialize it to `car.ser`, and deserialze it back.
8. Identify the error: `public void doMath() { throw new IOException(); }` (Hint: Check the method signature).
9. Write a stacked multiple catch block that catches `ArithmeticException` first, and the generic `Exception` second.
10. What keyword prevents a specific variable from being saved during object serialization?
11. Write a program that asks the user for an integer using Scanner. Catch `InputMismatchException` if they type text instead of numbers.
12. Explain why `try-with-resources` makes `finally` blocks mostly obsolete for closing files.
13. Write a loop that reads a file and uses a counter variable to determine how many lines the file has.

IT STUDY HUB

14. What happens if you try to catch `Exception` before catching `ArithmeticException`?
15. If a `return` statement is executed inside a `try` block, does the `finally` block still execute?

B. University Viva Questions & Answers

Q: What is an Exception in Java?

Ans: An exception is an unwanted runtime event that disrupts the normal execution flow of the program. It can be caught and handled.

Q: What is the difference between an Error and an Exception?

Ans: Errors (like OutOfMemoryError) indicate serious JVM or system-level problems that applications should not try to catch. Exceptions (like IOException) are program-level issues that can and should be caught and recovered from.

Q: What is the base root class for all errors and exceptions?

Ans: The 'Throwable' class.

Q: What is the difference between Checked and Unchecked Exceptions?

Ans: Checked exceptions are checked at compile-time, and the compiler forces you to handle them (e.g., IOException). Unchecked exceptions occur at runtime, are descendants of RuntimeException, and the compiler doesn't force handling (e.g., NullPointerException).

Q: What are the keywords used for exception handling?

Ans: try, catch, finally, throw, and throws.

Q: What is the purpose of the 'finally' block?

Ans: It executes code that MUST run regardless of whether an exception occurred or not. It is typically used to release resources like closing files or network sockets.

Q: Can we write a 'try' block without a 'catch' block?

Ans: Yes, but only if it is immediately followed by a 'finally' block. However, it will not handle the exception; it will just ensure the cleanup code runs before crashing.

Q: What is the difference between 'throw' and 'throws'?

Ans: 'throw' is a verb used inside the method body to manually trigger a specific exception object. 'throws' is a declaration in the method signature indicating that the method might result in certain checked exceptions, delegating the responsibility to the caller.

IT STUDY HUB

Q: How does Java resolve multiple catch blocks?

Ans: It checks them sequentially from top to bottom. It executes the first catch block whose exception parameter matches the thrown error. Therefore, child exceptions must be placed before generic parent exceptions.

Q: How do you create a custom exception?

Ans: By creating a class that extends 'Exception' (for checked) or 'RuntimeException' (for unchecked), and overriding the constructor to pass a custom message to the super() class.

Q: What is the 'File' class used for?

Ans: It represents a file or directory path name abstractly. It is used to check file existence, create empty files, delete files, or get file metadata (size, path), but it does not read/write the contents.

Q: What is the difference between a Byte Stream and a Character Stream?

Ans: Byte streams (InputStream/OutputStream) read/write raw 8-bit binary data (images, audio). Character streams (Reader/Writer) read/write 16-bit Unicode characters, making them ideal for text files.

Q: Why are BufferedReader and BufferedWriter preferred over basic FileReaders?

Ans: They utilize an internal buffer (memory cache). Instead of executing costly OS disk operations for every single character, they read/write large chunks of text into RAM at once, drastically improving performance.

Q: How does the 'append' mode work in FileWriter?

Ans: If the FileWriter constructor is passed 'true' as the second argument, it will add new text to the end of the existing file rather than overwriting and destroying the old data.

Q: What does BufferedReader.readLine() return when it reaches the end of the file?

Ans: It returns 'null'.

Q: What is Object Serialization?

Ans: The process of converting the state of a live Java object into a byte stream so that it can be persisted to a file or transmitted over a network.

Q: How do you make a class Serializable?

Ans: By making the class implement the 'java.io.Serializable' interface. This is a marker interface (contains no methods) that signals to the JVM that the object is approved for serialization.

IT STUDY HUB

Q: What does the 'transient' keyword do?

Ans: It prevents a specific variable from being serialized. When the object is deserialized later, the transient variable will initialize with its default value (e.g., null for Strings), keeping sensitive data like passwords secure.

Q: What is a 'try-with-resources' statement?

Ans: Introduced in Java 7, it allows resources (like BufferedReaders) to be declared within the parentheses of a try block. Java guarantees that these resources will be closed automatically when the block exits, eliminating the need for a finally block.

Q: What is a Stack Trace?

Ans: A diagnostic report printed to the console when an unhandled exception occurs. It shows the exact sequence of method calls that led to the error, helping developers pinpoint the exact line of code that crashed.

C. Beginner/Intermediate Interview Questions

Interview Q: Why is catching the generic 'Exception' class considered a bad practice in large applications?

Ans: Catching the generic 'Exception' acts as a blind safety net that catches everything, including bugs you didn't anticipate. This can silently swallow critical logical errors (like NullPointerExceptions), making debugging incredibly difficult. You should only catch specific exceptions you know how to handle.

Interview Q: If an exception is thrown inside a 'finally' block, what happens?

Ans: If the finally block throws an exception, it suppresses the original exception thrown in the try/catch block. This is dangerous because the original root cause of the crash is lost. Cleanup code in finally blocks should ideally be wrapped in its own safe try-catch.

Interview Q: Can you force the 'finally' block NOT to execute?

Ans: Yes, but only by terminating the JVM forcibly using 'System.exit(0)' inside the try or catch block. A normal 'return' statement will not stop the finally block from executing.

IT STUDY HUB

Interview Q: What is the difference between ClassNotFoundException and NoClassDefFoundError?

Ans: ClassNotFoundException is a Checked Exception that occurs when an application tries to load a class at runtime using its string name (e.g., Class.forName()), but the class is not found in the classpath. NoClassDefFoundError is a fatal Error that occurs when the JVM compiled successfully with the class present, but the class file was deleted or moved before runtime.

Interview Q: Explain why NullPointerException is Unchecked, but IOException is Checked.

Ans: A NullPointerException is almost always a programming logic flaw (forgetting to initialize an object). The compiler shouldn't force you to wrap every single object interaction in a try-catch. An IOException relies on external factors outside the programmer's control (network drops, hardware failure), so the compiler forces you to plan for the inevitable worst-case scenario.

Interview Q: When designing an API, when should you create a Custom Exception vs. using a standard one?

Ans: Use standard exceptions (IllegalArgumentException, IllegalStateException) if the error perfectly matches core programming flaws. Create Custom Exceptions (e.g., InsufficientFundsException) when the error represents a specific failure in your domain's business logic, allowing calling code to react to business rules specifically.

Interview Q: How does serialization handle object references? If object A contains object B, does B get serialized?

Ans: Yes, serialization navigates the object graph recursively. If Object A is serialized, the JVM automatically serializes Object B inside it, provided that Object B also implements the Serializable interface. If B does not, a NotSerializableException is thrown.

Interview Q: What is the serialVersionUID?

Ans: It is a unique version identifier assigned to a Serializable class. When deserializing an object, the JVM compares the UID of the file data with the UID of the current class definition. If you altered the class (e.g., added a field) and the UIDs mismatch, it throws an InvalidClassException to prevent data corruption.

Interview Q: If you are reading a 5GB text file, why is loading it into a String variable a terrible idea?

Ans: A String loads entirely into RAM. Attempting to load a 5GB String will exhaust the JVM heap memory and cause an OutOfMemoryError. You should use a BufferedReader to read and process the file line-by-line, keeping RAM usage minimal.

IT STUDY HUB

Interview Q: What is the fundamental difference between Java NIO (New I/O) and standard java.io?

Ans: Standard java.io uses blocking, synchronous streams (the thread pauses while waiting for data to read/write). Java NIO introduces non-blocking, asynchronous channels and buffers, allowing a single thread to manage thousands of active file/network connections simultaneously, critical for high-performance servers.

IT STUDY HUB

10. Module Summary and Key Terms

Key Terms Glossary

- **Exception:** A recoverable runtime error that disrupts standard code execution.
- **Stack Trace:** The diagnostic log printed to the console detailing exactly where a crash occurred.
- **Checked Exception:** An exception validated by the compiler; must be handled or declared.
- **Unchecked Exception:** A logic error (RuntimeException) not validated by the compiler.
- **Stream:** A sequence of data flowing from a source (Input) or to a destination (Output).
- **Serialization:** Converting a Java object's state into a binary byte stream for storage.

Module Summary

In this module, you transitioned from writing fragile code to robust software. You learned how to anticipate failures using **Exceptions**, intercepting them safely with `try-catch-finally` blocks. You mastered the hierarchy of errors, understood the difference between Checked and Unchecked exceptions, and established business rules by generating Custom Exceptions using `throw` and `throws`.

We then escaped the temporary constraints of RAM by mastering **File Handling**. You learned to navigate the file system using the `File` class, and employed high-performance `BufferedReader` and `BufferedWriter` tools to seamlessly inject and

extract data from the physical hard drive. Finally, we explored Object Serialization to freeze live objects in time.

WHAT COMES NEXT?

MODULE 7 — COLLECTIONS FRAMEWORK & GENERICS

Arrays are powerful, but their fixed size is a severe limitation. If you don't know how many students will register, an Array will either run out of space or waste memory. In the next volume, we will unlock Java's most powerful data structures: **The Collections Framework**. You will master dynamic `ArrayLists`, unique `Sets`, and key-value `HashMaps`, heavily utilized in all modern software engineering.

IT STUDY HUB SERIES